Lecture 10 Scribe

lecture_scribe_structure_only.pdf
PDF

L10_S2.pdf
PDF

Using I same structure above from lecture scribe structure file , generate a lecture scribe for the content L10 file . I want this scribe as such that I can refer it for my exams final revision and I don't refer to any other lecture notes just this scribe. Use all the same notations and formats as given in L10 file. Don't add any other extra content. Also just use lecture scribe structure file only for the structure. No extra additions are expected from your side.

Below is the Lecture Scribe for **L10** strictly using the structure from the lecture scribe structure file and content only from L10 .
No extra content has been added.

Lecture 10: Randomized Min-Cut Algorithm

---

1. Overview

- Min-Cut Problem
- Why use min-cut?
- What is min-cut?
- Successfull Min-cut run
- Unsuccessfull Min-cut run
- Max-Flow Min-Cut Theorem
- Deterministic Min-Cut Algorithm
- Stoer–Wagner minimum cut algorithm
- Psuedocode
- Randomized Min-Cut Algorithm
- Why randomized algorithm?

- Karger's Randomized Algorithm
- Psuedocode
- Comparision: Deterministic Min-Cut vs Randomized Min-Cut Algorithm
- Theorem for min-cut set
- Python Simulation

---

2. Definitions and Notation

Cut-set:
A cut-set in a graph is a set of edges whose removal breaks the graph into two or more connected components.

Min-Cut Problem:
Given a graph $G = (V, E)$ with n vertices, the minimum cut (min-cut) problem is to find a minimum cardinality cut-set in G.

Edge Contraction:
The main operation in the algorithm is edge contraction, which removes an edge from a graph while simultaneously merging the two vertices.

In contracting an edge (u, v):

- Merge vertices u and v into one vertex.
- Eliminate all edges connecting u and v.
- Retain all other edges in the graph.
- The new graph may have parallel edges but no self-loops.

Flow Network Terms:

- Capacity of a cut = sum of capacities of edges oriented from vertex $\in X$ to vertex $\in Y$.
- Minimum cut = cut with smallest possible capacity.
- Minimum cut capacity = capacity of minimum cut.
- Maximum flow = largest possible flow from source S to sink T.

---

3. Assumptions and Conditions

- Graph $G = (V, E)$ is given.
- For randomized algorithm, graph is treated as an undirected multigraph.

- Randomized min-cut algorithms are sensitive to initial edge choices.
- If critical edges are contracted early, algorithm may not find minimum cut.
- Randomized algorithms provide probabilistic guarantees.

---

4. Main Results / Theorems

Max-Flow Min-Cut Theorem

"In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut."

Stoer–Wagner Theorem

Let s and t be two vertices of graph G. Let $G/\{s, t\}$ be the graph obtained by merging s and t.

A minimum cut of G is the smaller of:

- A minimum $s - t$ cut of G, or
- A minimum cut of $G/\{s, t\}$.

Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least:

$$\frac{2}{n(n-1)}$$

---

5. Proofs / Derivations

Stoer–Wagner Justification

Either:

- There exists a minimum cut separating s and t, in which case minimum $s - t$ cut is the minimum cut of G,

or

- No such separation exists, and minimum cut of $G/\{s, t\}$ gives the answer.

6. Worked Examples

Successful Min-Cut Run

A successful min-cut run refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

Unsuccessful Min-Cut Run

An unsuccessful min-cut run refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

Why Randomized Algorithm?

Randomized algorithms:

- Provide probabilistic guarantee of success
- Provide more accurate estimate with fewer iterations
- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness

Deterministic Min-Cut (Stoer–Wagner)

Time Complexity:

$$O(V \cdot E + V^2 \log V)$$

- Guarantees exact minimum cut
- Higher time complexity for large graphs

Randomized Min-Cut (Karger's Algorithm)

Time Complexity:

$$O(V^2)$$

- Provides approximate minimum cut with high probability

Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

- $A \leftarrow \{a\}$
- while $A \neq V$
  - add to A the most tightly connected vertex
- return cut weight as “cut of the phase”

Algorithm 2: MinimumCut(G)

- while $|V| \geqslant 1$
  - choose any $a \in V$
  - MinimumCutPhase(G, a)
  - if cut-of-the-phase is lighter than current minimum cut
    - store it
  - shrink G by merging last two added vertices
- return minimum cut

Algorithm 3: Recursive-Randomized-Min-Cut(G, α)

Input: Undirected multigraph G with n vertices, integer $\alpha > 0$

Output: Cut C of G

If $n \leqslant \alpha^3$:

- $C \leftarrow$ min-cut of G using brute force

Else:

- for $i = 1$ to α:
  - Apply $n - \frac{n}{\sqrt{\alpha}}$ random contraction steps
  - $C' \leftarrow$ Recursive-Randomized-Min-Cut(G', α)
  - If $i = 1$ or $|C'| < |C|$:
    - $C \leftarrow C'$

Return C

7. Logical Dependencies / Summary

- Min-cut identifies weakest connectivity in a graph.
- Edge contraction is core operation.
- Deterministic approach guarantees exact solution.
- Randomized approach improves efficiency with probabilistic guarantees.
- Max-flow equals min-cut capacity in flow networks.
- Karger's algorithm outputs min-cut with probability at least $\frac{2}{n(n-1)}$.

End of Lecture 10 Scribe.